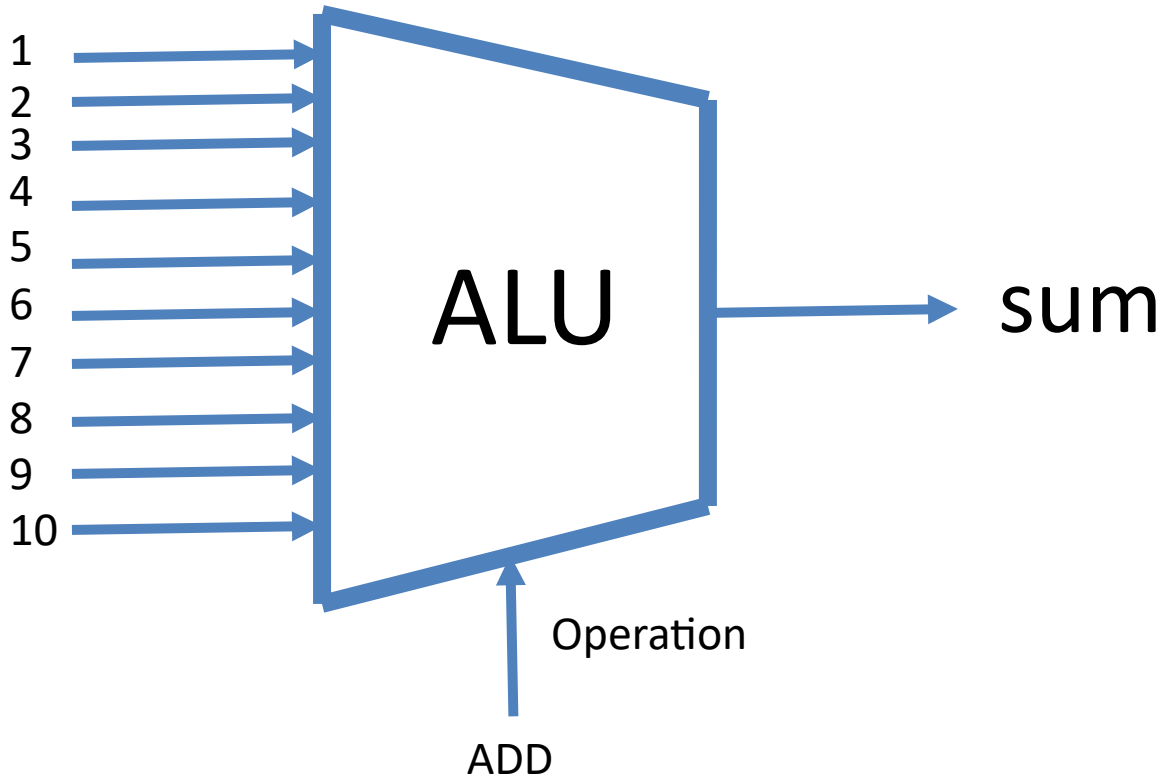
Variables

Let's Review Memory

We need to remember a calculation or state

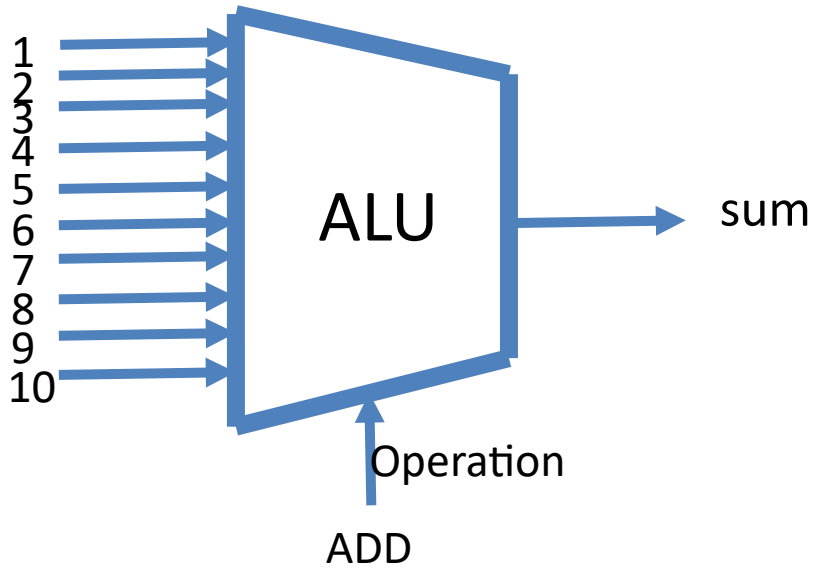


Memory

Physically have to put the results of the add somewhere

That place is called the address of sum

It is the location of a memory device



Location of Storage

Address is the location of memory or symbol (variable)

The computer knows them as unsigned integers

What are integers and unsigned integers?

Let’s review

Everything Is Bits

A memory device holds one of **two** *states*

We call them **0** *and* **1**

One such device is a **bit,** *a* **bi***nary digi***t**

n bits give **2^n** *different patterns*

4 bits give 16 patterns

Sixteen Patterns

0000	*0001*	*0010*	*0011*
0100	*0101*	*0110*	*0111*
1000	*1001*	*1010*	*1011*
1100	*1101*	*1110*	*1111*

Sixteen patterns. Nothing here says what any of them **means**.

Meaning Is a Mapping

A pattern by itself means **nothing**

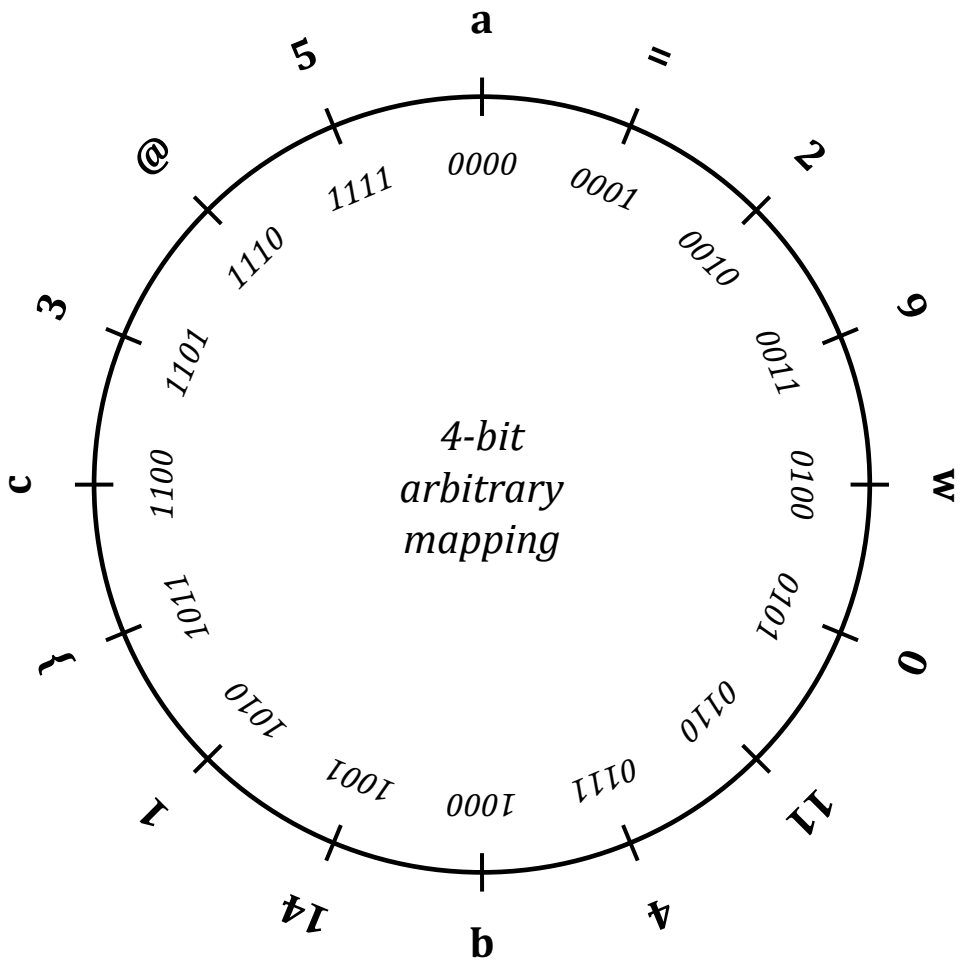
We must **choose** *an association for each pattern*

Any one-to-one mapping is legal

The hardware does not care but **we** *do*

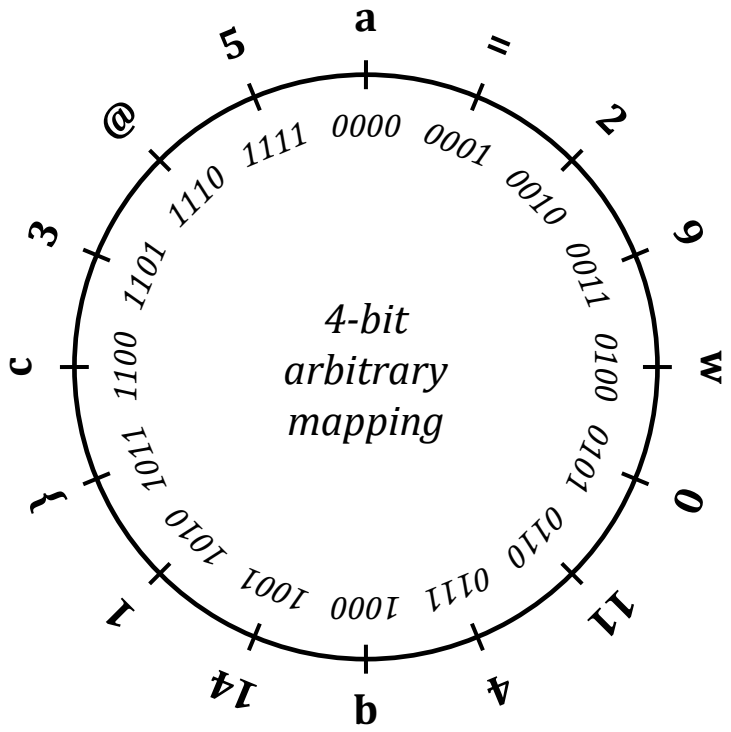
Some associations are more efficient than others

Bit patterns to symbols



Make an association or 1 to 1 mapping

An Arbitrary Mapping



Legal - every pattern has one value
Not efficient for arithmetic
Requires a complex transformation
***1 + 1** lands nowhere useful or obvious*

Why an Arbitrary Mapping Fails

No relation between the **pattern** *and the* **value**

Adding 1 could move anywhere on the circle

Every operation needs a **lookup table**

4 bits	*16 entries*
64 bits	*18 billion billion entries*

We need a mapping that **computes**

Positional Notation

$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
1	*0*	*1*	*1*

$$1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1$$

$$= \mathbf{11}$$

*Each column has a **weight** - a power of two*

The Unsigned Rule

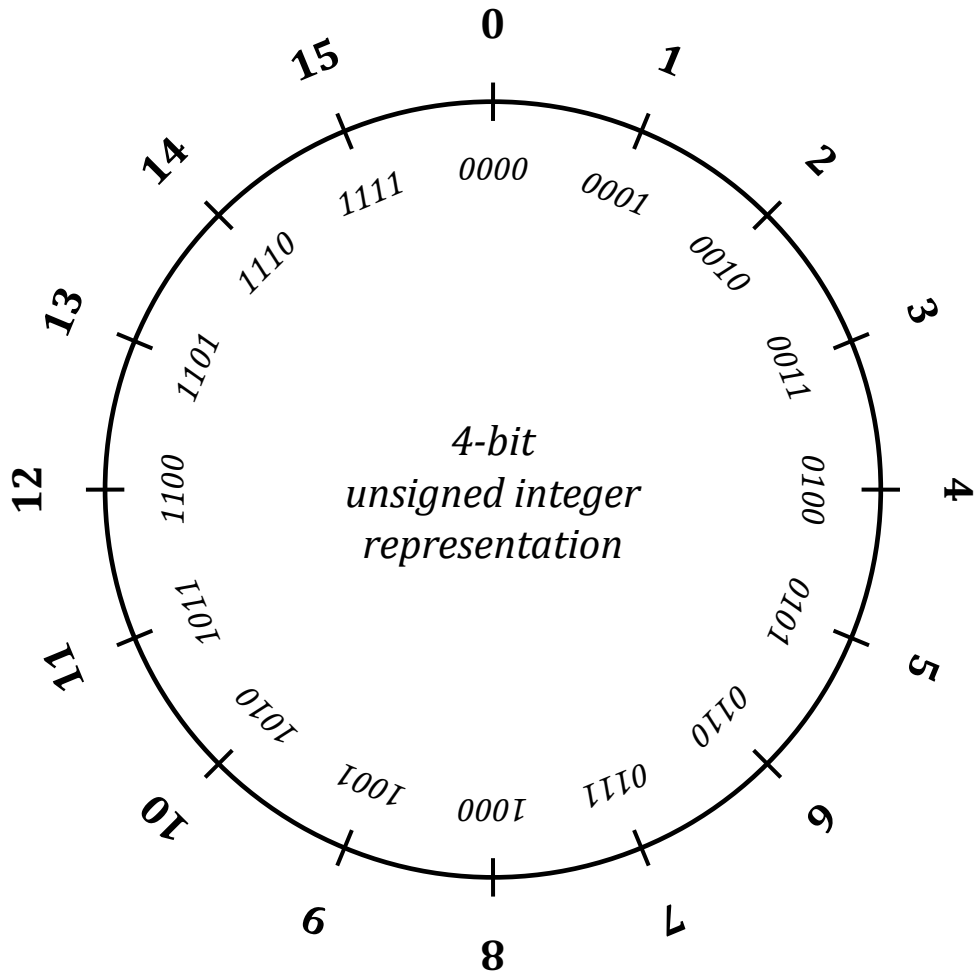
Number = sum of the **weights** *whose bit is 1*

$$b3 \cdot 2^3 + b2 \cdot 2^2 + b1 \cdot 2^1 + b0 \cdot 2^0$$

1011	*= 8 + 0 + 2 + 1*	*= 11*
1111	*= 8 + 4 + 2 + 1*	*= 15*
0000	*= 0*	*= 0*

One rule, any number of bits

Unsigned Representation



Neighbouring patterns are neighbouring numbers

Counting Works

Add 1 = move **one tick clockwise**

The bit pattern and the value stay in step

Past the top: *1111 + 1 = 0000*
15 + 1 = 0

The count **wraps** - *arithmetic modulo 2^n*

The circle is not a mistake; it is the machine

A computer only has a finite bit length

Unsigned Range

Bits	Patterns	Range
8	*256*	*0 .. 255*
16	*65,536*	*0 .. 65,535*
32	*4,294,967,296*	*0 .. 4,294,967,295*
64	1.8×10^{19}	$0 .. 1.8 \times 10^{19}$

n bits: 0 .. 2^n – 1

Addresses Are Unsigned

An address is a **position**, *never negative*

Unsigned gives us:

the **largest** *range for the bits we have*

32 bits	*4 GB of address space*
64 bits	*more than we can physically build – 18 exa Bytes*

We Also Need Negatives

Program data is often **signed**

Still only 16 patterns for 4 bits

So some patterns must mean **negative**

Whatever we choose, we **lose** *positive range*

Ideally, we would like to cut it in half

Half positive, half negative but zero spoils the fun

Sign and Magnitude

The obvious idea: the top bit is the **sign**

0 101 = +5
1 101 = –5

Problems:

0000 = +0 and 1000 = –0

Two *zeros - which one do you compare?*

The adder needs special cases needs to handle both zeros

Resulting in more complicated and larger hardware

Biased (Excess-K)

Another way:

Read the pattern as **unsigned**, *then subtract a fixed* **bias** *K*

K is usually half the full unsigned range

either $(2^n / 2)$ *or* $(2^n / 2) - 1$

$$value = pattern - K$$

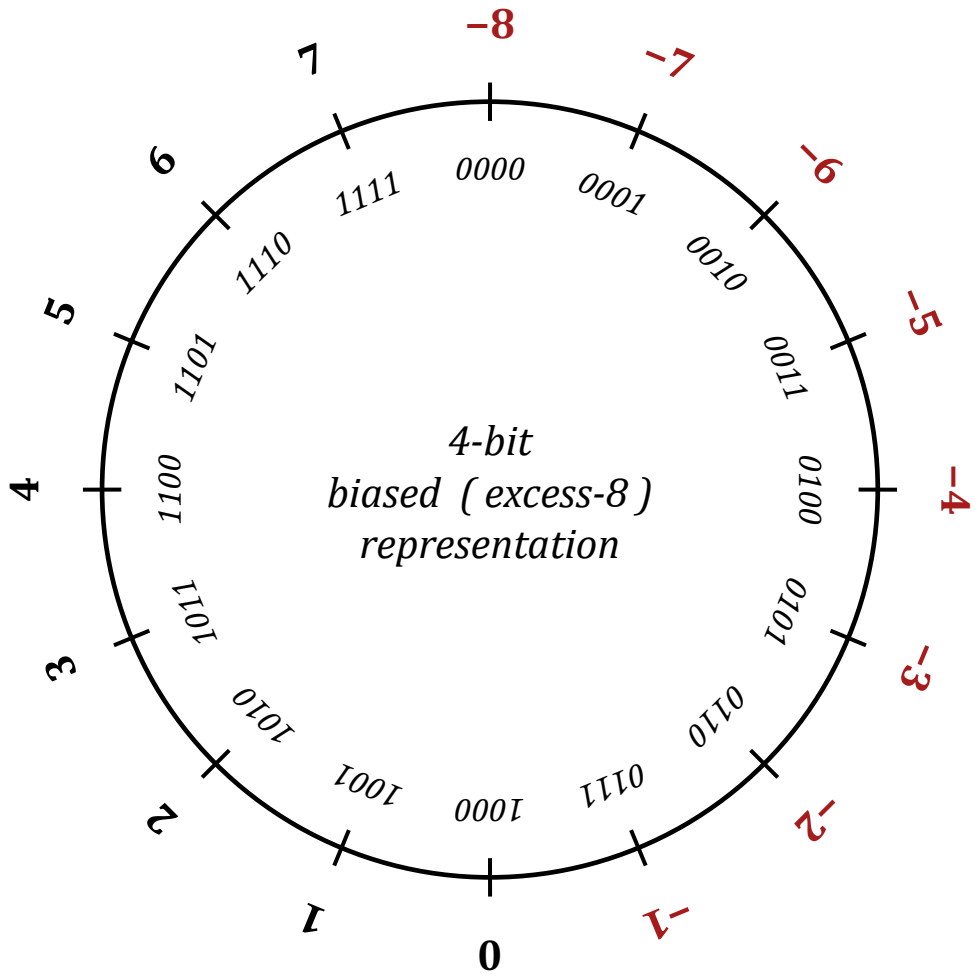
4 bits with K = 8:

0000 = −8

1000 = 0

1111 = +7

Biased Representation



value = pattern – 8 - the patterns now run in ***numeric order***

Why Bias Is Useful

The **unsigned order** *is the* **signed order**

0000 < 0001 < … < 1111
−8 < −7 < … < +7

So an **unsigned compare** *sorts signed values*

Used for the **exponent** *of a floating point numbers:*

K = 127 for f32, 1023 for f64

The Range Is Not Symmetric

n bits: *– 2^(n–1) .. + 2^(n–1) – 1*

4 bits: *–8 .. +7*

There are eight negatives but only seven positives because **0** *takes one of the positive patterns*

–8 has **no positive partner***: negate 1000 and you get 1000 back*

One's Complement

Another try:

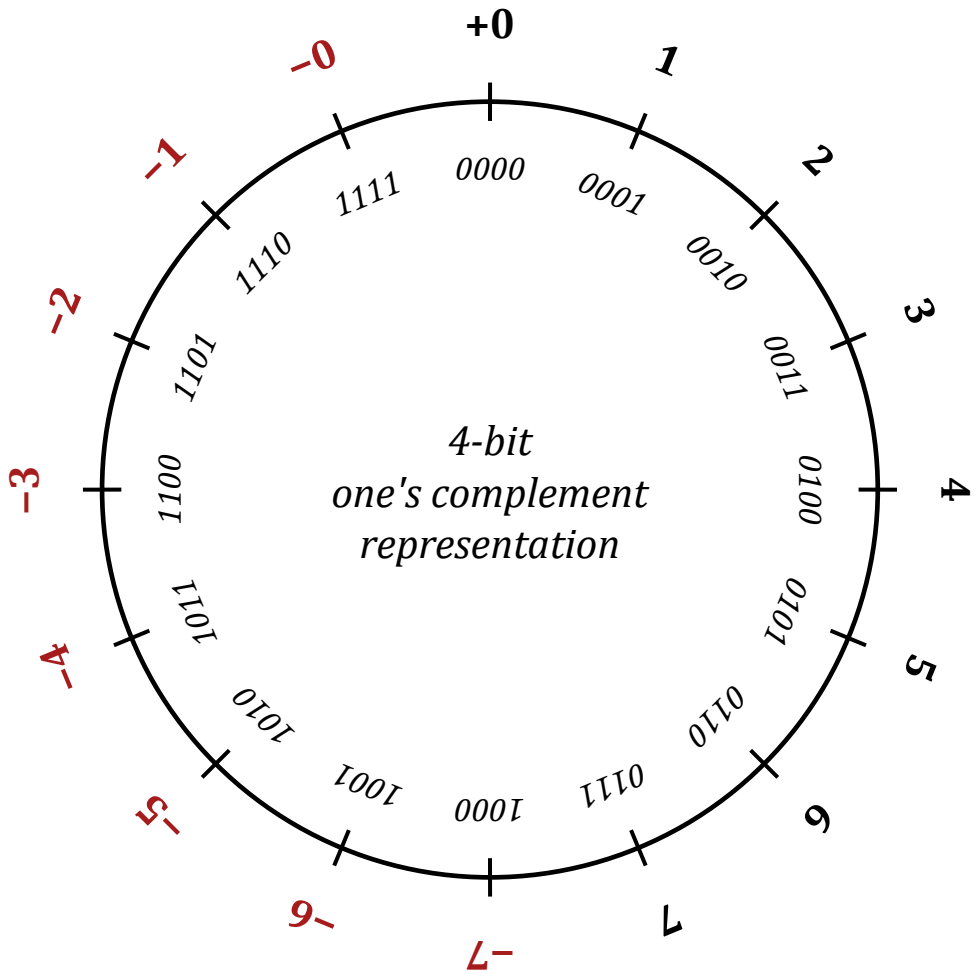
Negate by **flipping every bit**

0011 = +3
1100 = −3

Positive numbers are unchanged

The top bit still tells you the sign

One's Complement Representation



*Negate by flipping the bits - but **0000** and **1111** are both zero*

One's Complement: The Problems

Two zeros *0000 and 1111*

a == 0 needs two tests

End-around carry

a carry out of the top must be added back in at the bottom



Example 5 – 2 = 5 + (-2) = 0101 + 1101 = 1 0 0 1 0 => 0 0 1 1

Correct, but the hardware is awkward and complex

Two's Complement

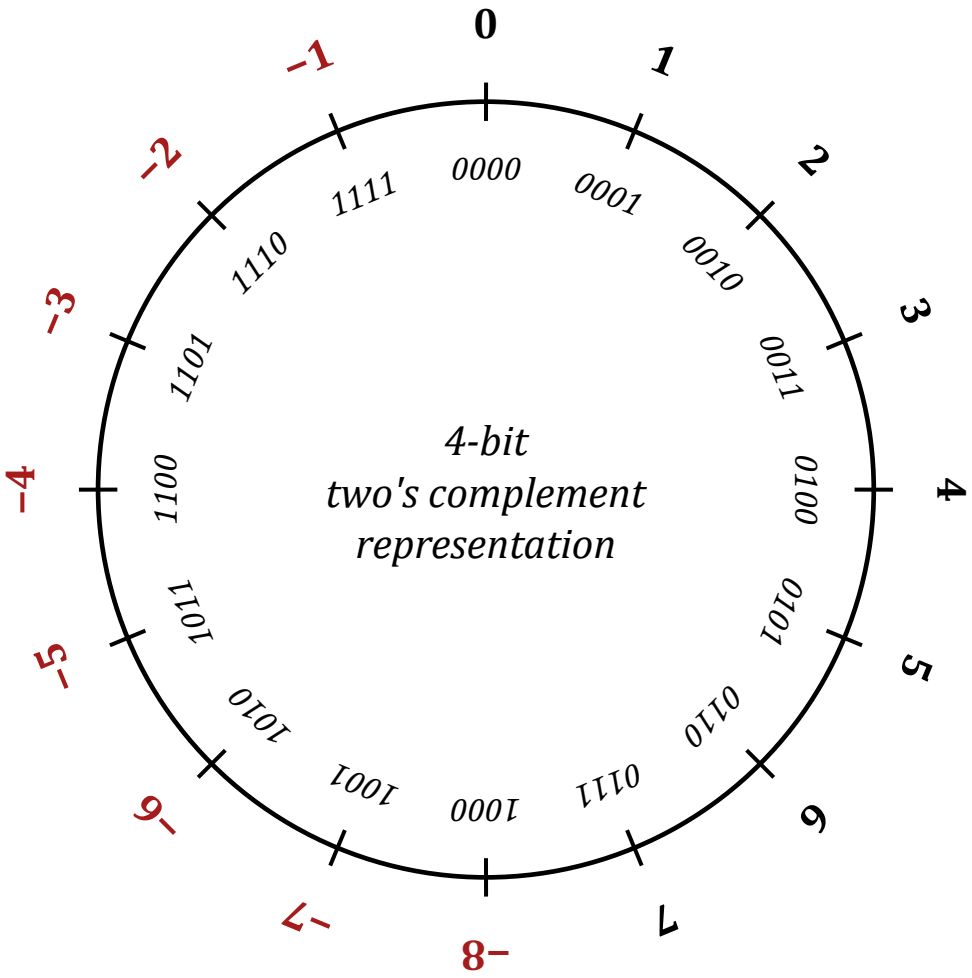
Negate by **flip every bit, then add 1**

	0101	*= +5*
flip	*1010*	
add 1	*1011*	*= −5*

Do it again and you get 0101 back

This is what every modern machine uses

Two's Complement Representation



*One zero, and the top bit is 1 for exactly the **red** half*
The ambiguity is shifted to - / + 8

Why Two's Complement Wins

One zero *0000, and nothing else*

One adder *the unsigned adder just works*

One sign test *is the top bit 1?*

No carry fix-up *discard the carry out*

Subtraction becomes $a + (-b)$

Reading a Signed Number

– 8	**+ 4**	**+ 2**	**+ 1**
1	*0*	*1*	*1*

– 8 *0* *+ 2* *+ 1*

= **–5**

Same rule as unsigned - only the ***top weight is negative***

Signed and Unsigned Ranges

Bits	Unsigned	Two's complement
8	*0 .. 255*	*−128 .. +127*
16	*0 .. 65,535*	*−32,768 .. +32,767*
32	$0 .. 4.29 \times 10^9$	$-2.15 \times 10^9 .. +2.15 \times 10^9$
64	$0 .. 1.8 \times 10^{19}$	$-9.2 \times 10^{18} .. +9.2 \times 10^{18}$

Signed buys the low half by giving up the high half

The Same Bits, Two Meanings

Pattern	Unsigned	Two's complement
0000	*0*	*0*
0111	*7*	*+7*
1000	*8*	*−8*
1011	*11*	*−5*
1111	*15*	*−1*

The bits are identical. Only the **declared type** *differs.*

Binary Is Hard to Read

A 28-bit address in binary:

00011111100010011001101111100

Count the bits. Now find the error.

We need a shorter way to write the **same** *bits*

Hexadecimal

Base **16** *- digits 0 1 2 3 4 5 6 7 8 9 A B C D E F*

One hex digit = **exactly 4 bits**

So converting is just **grouping***:*
no arithmetic, no division

Four bits is one **nibble***; two nibbles are a* **byte**

The Sixteen Digits

Dec	Bits	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7

Dec	Bits	Hex
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

A = 10, B = 11, C = 12, D = 13, E = 14, F = 15

Reading an Address in Hex

0001 1111 1000 1001 1001 1011 1100

0001	*1111*	*1000*	*1001*	*1001*	*1011*	*1100*
1	*F*	*8*	*9*	*9*	*B*	*C*

= 0x1F899BC

*Group the bits by four **from the right**, then name each group*

One Byte, Two Digits

8 bits = 2 hex digits

0x00	*= 0*	
0x0F	*= 15*	
0x80	*= 128*	*(or −128 signed)*
0xFF	*= 255*	*(or −1 signed)*

Memory dumps are written this way

Writing Numbers in Code

	C / C++	**Rust**
decimal	*255*	*255*
hex	*0x1F*	*0x1F*
binary	*0b1011 (C23)*	*0b1011*
octal	*017*	*0o17*
readable	*1'000'000*	*1_000_000*

All of these are the **same bits**

Printing in Hex

C: *printf ("%x %X %#010x\n", v, v, v) ;*

C++: *std::cout << std::hex << v ;*

Rust: *println! ("{:x} {:#06x} {:#b}", v, v, v) ;*

The debugger will show you hex too:

(gdb) p/x v

(gdb) x/16xb &v

What to Remember

Bits are **patterns**; *meaning is a* **choice**

Unsigned	*positional powers of 2*
Two's complement	*flip and add 1*
Hex	*four bits per digit*

Now back to memory and addresses

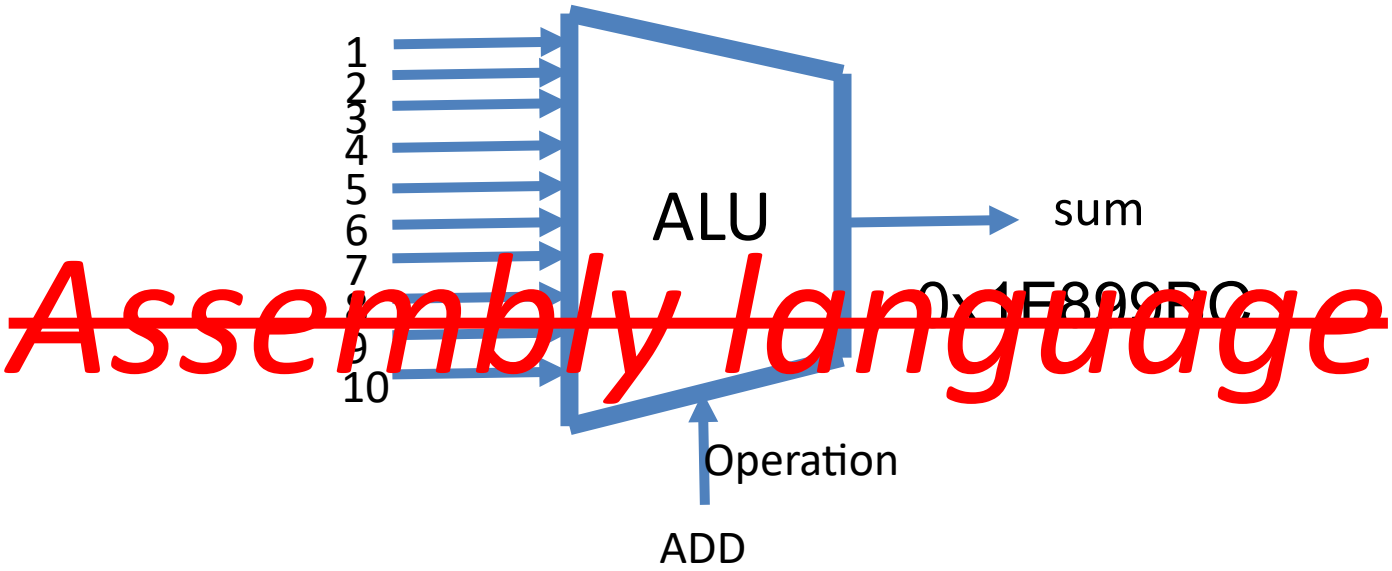
Location of Storage

Address is the location of memory or symbol (variable)

The computer knows them as unsigned integers

Unsigned integers start at 0 and go to $2^{numbits} - 1$ in binary

Too difficult for humans to keep track of these numbers



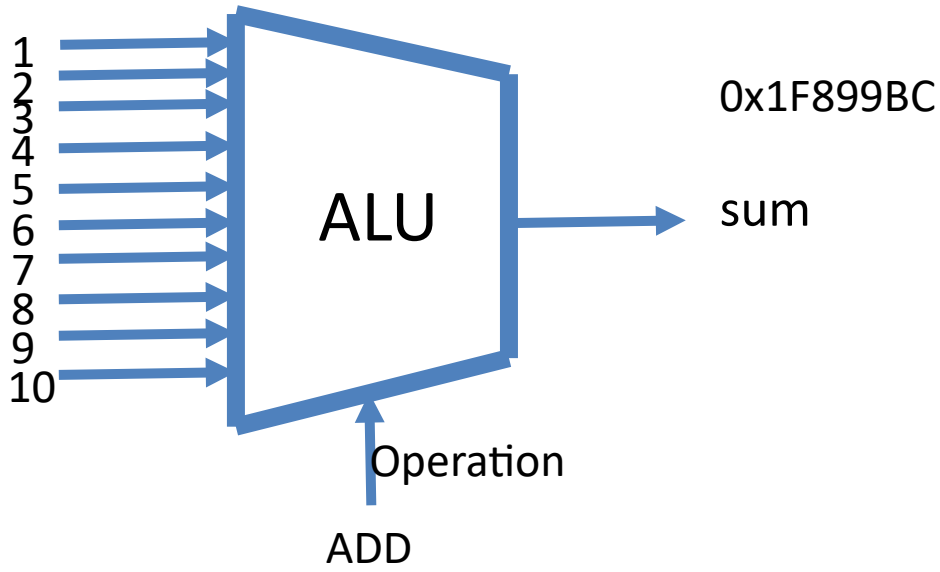
~~Assembly language~~

Symbol

To make it easier, we associate a symbolic name with the address

This is known as a symbol

Loosely called a variable



But not the value

The symbol is not the computed value

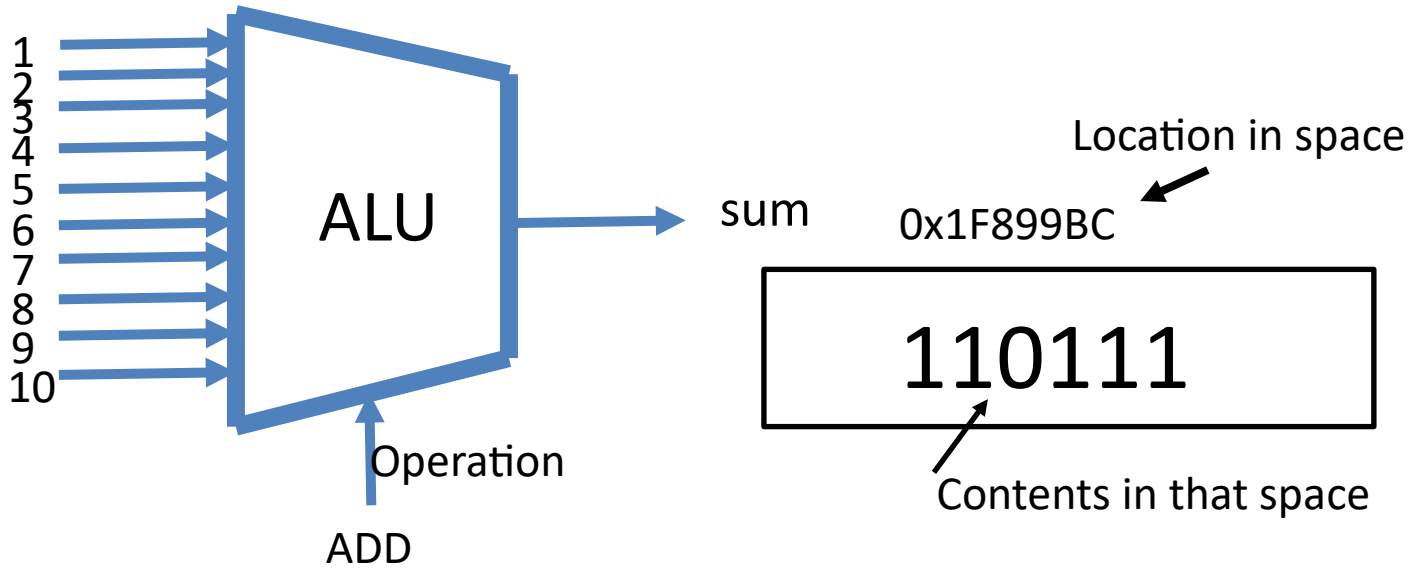
The address is a location

We need the contents of the address

The contents of sum is 55

The address is 0x1F899BC

Notice address is now written in hex due to size and convention



Making an association

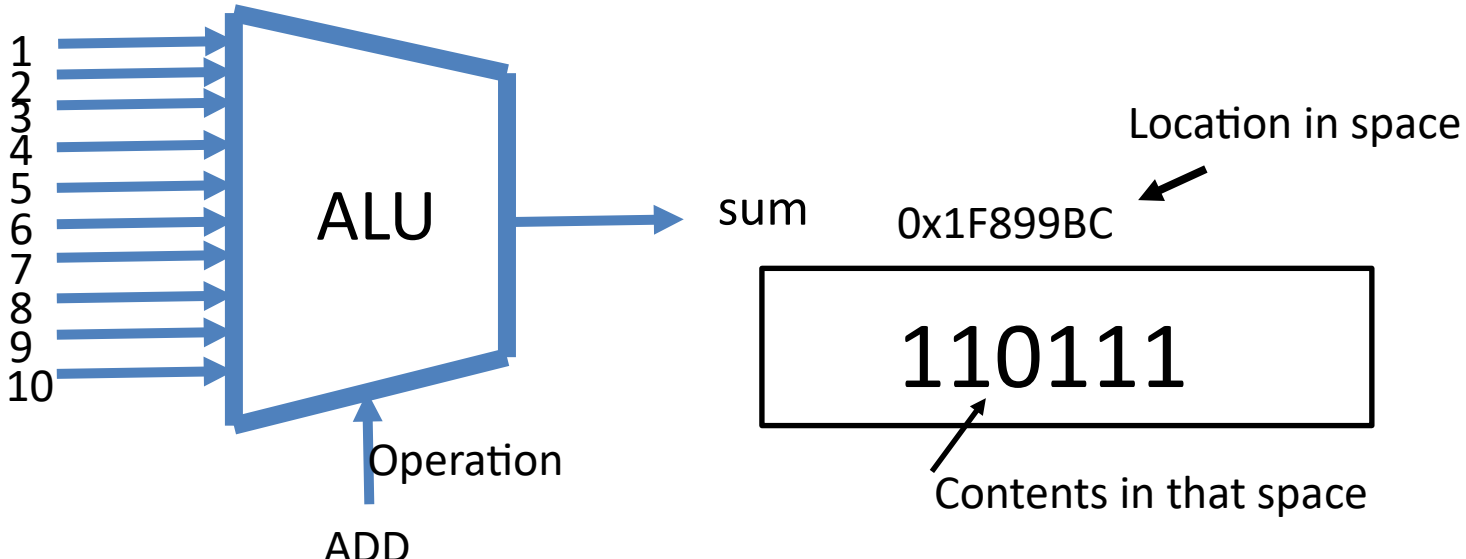
Symbolically make an association.

Let

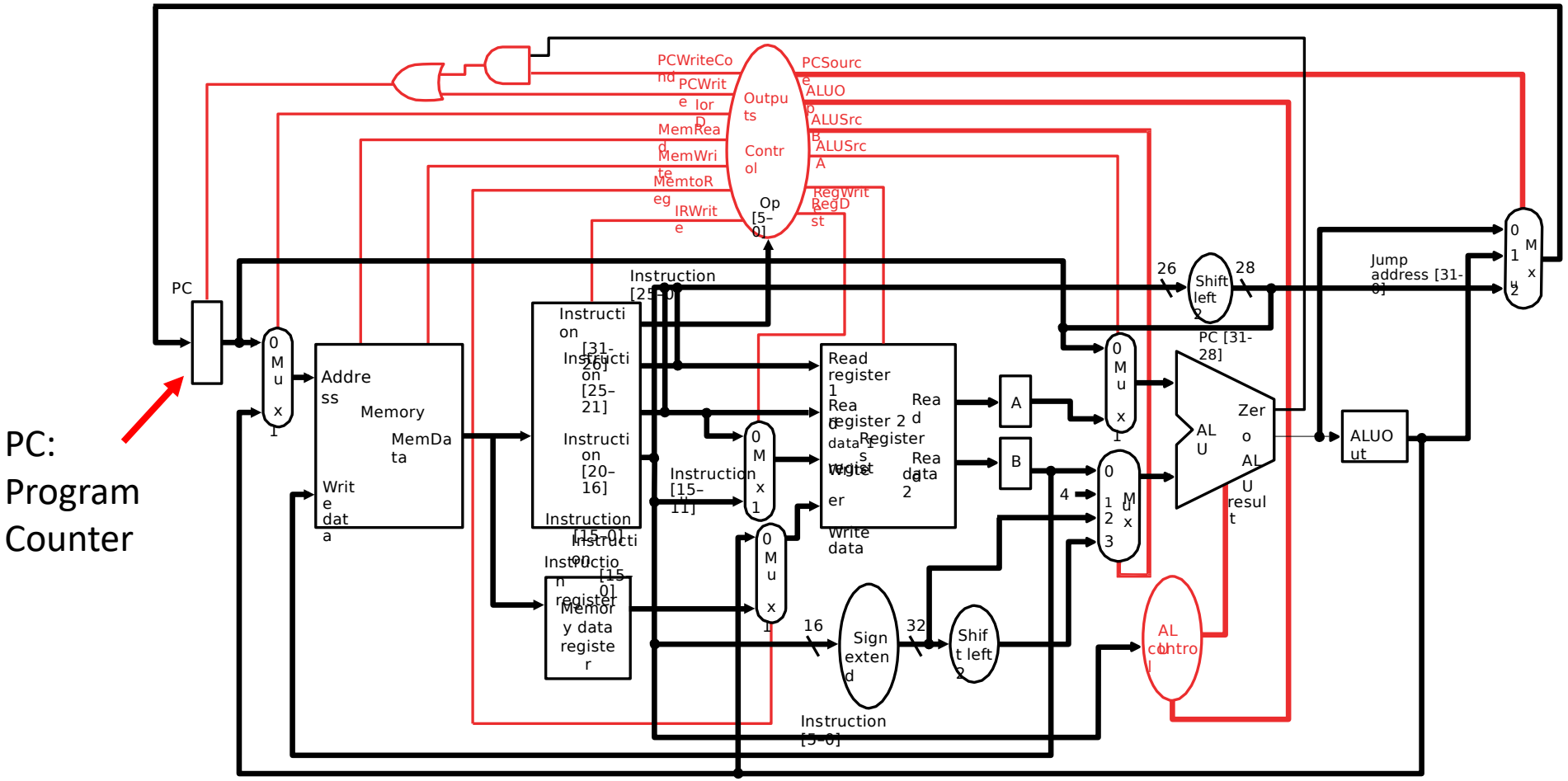
sum be the address or location of memory

$sum be the contents of memory

Very clean and exact



Associations in Computer Architecture



PC: Program Counter

The PC or Program Counter is an address. It is the address of a register holding the address of the next instruction to be executed.

Aside: It is the hardware equivalent of a software pointer which will discuss in the next lesson

Making an association

Symbolically make an association.

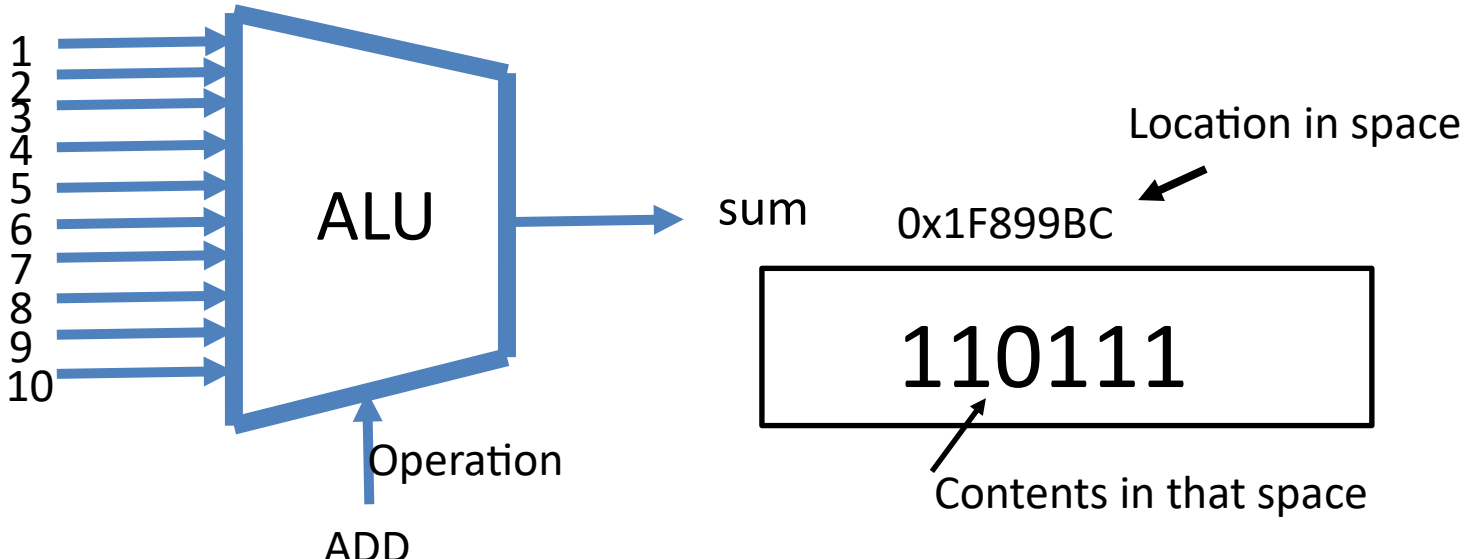
Let

sum be the address or location of memory

$sum be the contents of memory

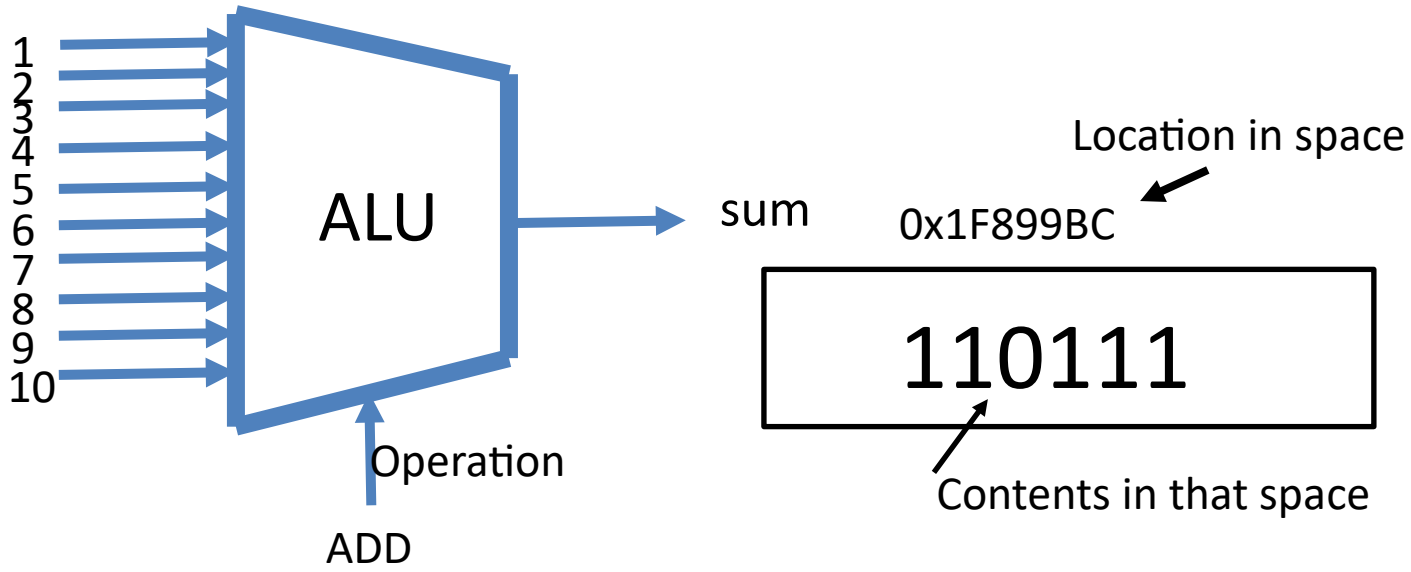
Very clean and exact

Some languages do it this way! But not C, C++, or Rust!



Humans hate to type

Most of the time we want to process data
We really don't want to care about the location
We just want the answer
So just drop the address from consideration
The symbol represents the contents of a memory location



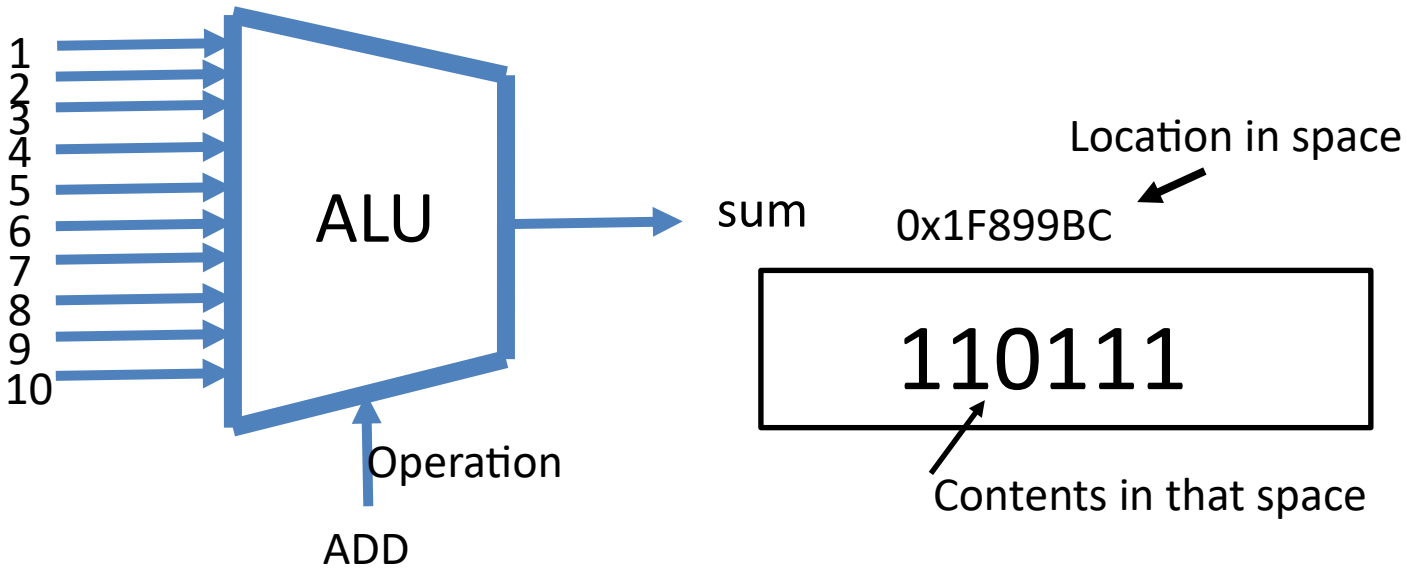
Humans hate to type

If we want the address, we will use the & address operator

&sum is 0x1F899BC

sum is 11011 in binary or 55

C, C++, and Rust all do it this way



Machine Size

The implemented number of bits of the computer's data bus

The natural size of the computer

Arithmetic Logic Unit works on this many bits at one time

Computer is most efficient with this data size

Smaller is usually just as fast but bigger takes more time

Address Space

Since the data bus (num_databus_bits) is used to compute addresses

address size : [0 ... $2^{num_databus_bits} - 1$]

The universe of addresses representable is called the

address space

For 32 bits

address space : [0 … $2^{32} - 1$] = 4,294,967,295 = 4 GBytes

For 64 bits

address space : [0 … $2^{64} - 1$] =

18,446,744,073,709,551,615 = 18.45 ExaBytes

Implementing Address Space

Computer does not have to implement the entire address space

The physical address space may be smaller than the full address space

Embedded Systems which utilize 32 bit MCUs do not usually implement all of the possible 4 G Byte address space

Desktops, Laptops, and Notebooks do not implement all 18 Exa Bytes

Virtual Memory

Every access to memory ultimately needs a physical address

To make things easier for the programmer, present the allusion that memory is just one large contiguous set of addresses

This allusion is called **virtual memory**

Memory Mapping

This requires a translation:

virtual address → physical address

A computer needs to perform the translation and check to see if the physical address exists

Modern computers have hardware to do this efficiently

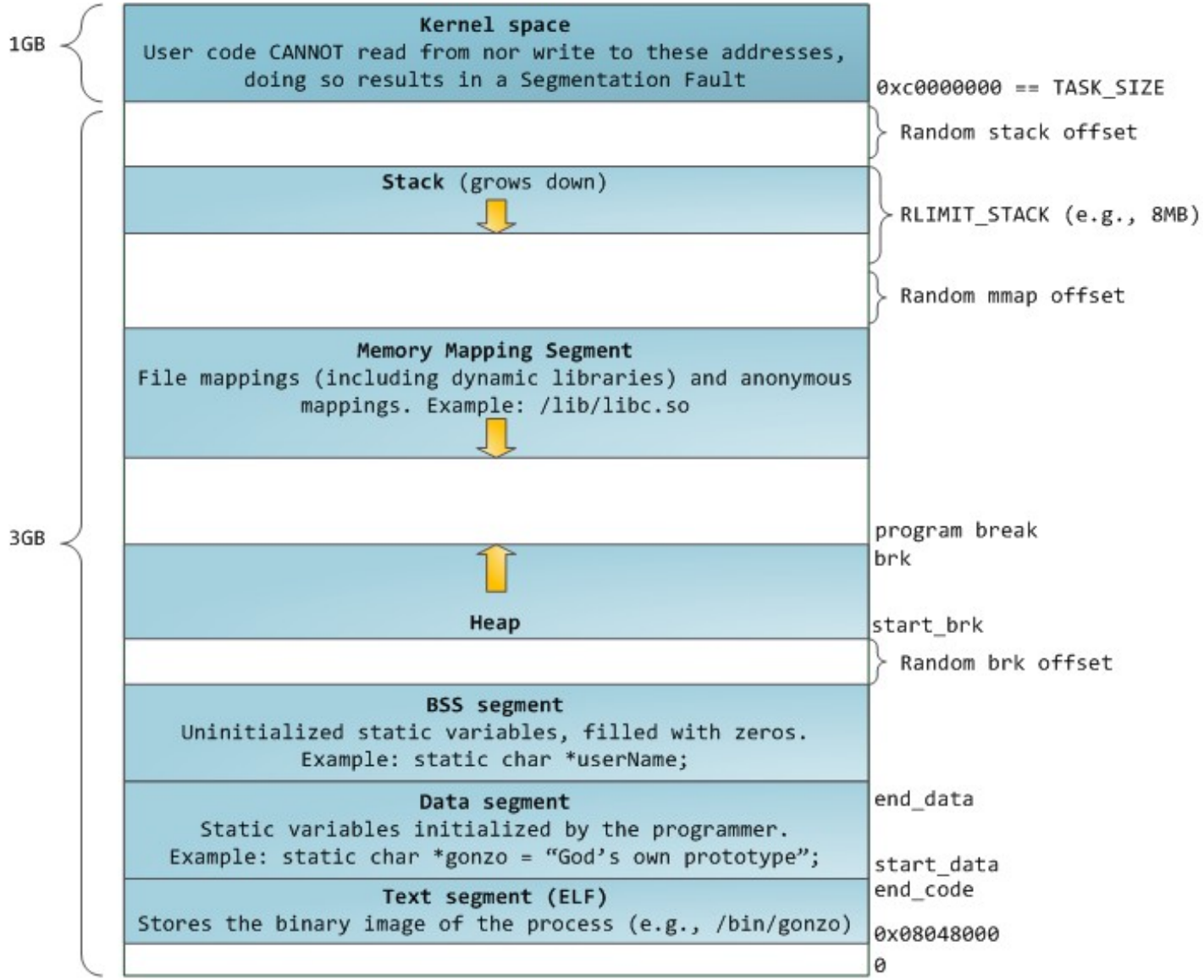
Users see one large space; physical space can be segmented

Allows computer architect to optimize variable placement

Aside: Hardware is named the memory mapper and may use special memory used to speed the process called the TLB or translation look aside buffer.

Linux Memory Mapping

Memory map of Linux 32 bit OS



Addresses are Unsigned Integers

Every memory access needs to be checked

Unsigned integers need only one check : limit of address space

Integers would require two checks! One for positive limit and one for negative limit

Speed rules!

Common Machine Sizes

Most new desktop, laptop, and notebook computers are 64 bits

Most embedded computers have a 32 bit data and address space

It is best to use the machine word size because it is efficient

While addresses are unsigned integers, our program data can be either signed or unsigned

Natural Size on 64 Bit Machines

Rust :

i64 : signed integer $[-2^{63} \ldots 2^{63} - 1]$

u64 : unsigned integer $[0 \ldots , \ 2^{64} - 1]$

C, C++:

long long : signed integer $[-2^{63} \ldots 2^{63} - 1]$

unsigned long long : unsigned integer $[0 \ldots , \ 2^{64} - 1]$

Problematic! These languages existed before 64 bit machines

Use the C, C++ Preprocessor

The C and C++ languages use a preprocessor before they compile your code

The preprocessor is sensitive to lines that begin with **#**

Lines beginning with **#** are preprocessor directives

Meta language that controls how things are presented to the real compiler

Allows text to be substituted, deleted, and included

Use the C, C++ Preprocessor

The preprocessor allows different code to be compiled based on computer model differences

The preprocessor allows macro and variable definitions

Preprocessor Directives

The preprocessor has the following directive categories

- Macro Substitution
- File Inclusion
- Conditional Compilation
- Error Handling & Diagnostics
- Miscellaneous Directives

Macro Substitution Directives

#define `<macro_symbol[s]> <macro_definition>`

Creates a macro

Symbolic constant or a function-like block

The preprocessor replaces every occurrence of the macro name with the defined text

Example: (written text code)

```
#define int64_t   long long

int64_t my_variable ;
```

Macro Substitution Directives

Example: (compiler sees)

long long my_variable ;

Macro Substitution Directives

#undef *<macro_symbol>*

Cancels a previously defined macro

Stops replacement

Example:

#undef *int64_t*

After this statement, there is no further substitution

File Inclusion Directives

#include “filename”
#include <filename>

Includes the contents of a file
First form searches current directory
Second form searches a list of directories specified on command line as well as a default set of directories

Example:
#include <stdio.h>

Search in all default directories for a file called stdio.h and include it

Conditional Compile Directives

#if *<condition_statement>*

Checks a condition; compiles the following block if the condition evaluates to true.

#elif *<condition_statement>*

Acts as "else if" to check an alternate condition if the previous #if failed.

#else *<condition_statement>*

Provides a fallback block that compiles if all preceding conditions fail.

#endif

Marks the end of a conditional compilation block (#if, #ifdef, or #ifndef).

#ifdef *<condition_statement>*

Checks if a macro is defined, compiling the block only if it exists.

#ifndef *<condition_statement>*

Checks if a macro is not defined, commonly used as a header guard to prevent duplicate inclusions.

Include Only Once Directives

Example: In the file my_include.h

#ifndef *MY_INCLUDE_FILE_H*
#define *MY_INCLUDE_FILE_H*

#define *int64_t long long*

#endif

Include Only Once Directives

Example: In the file code main.c

#include "my_include.h"

#include "my_include.h"

Will only have one definition of int64_t as first include will define MY_INCLUDE_FILE_H

Necessary because multiple include files can have references to the same include file. Must have only one definition!

Error Handling Directives

These help generate compiler messages or force failures during build time

#error <output_message>

Outputs error

Stops the build

#warning <output_message>

Generates warning message

Does no halt the compilation process

Example:

error “Please fix <bits/endian.h>”

Miscellaneous Directives

#pragma <compiler directive>

Compiler specific directive (parallel commands)

#line <file and line number>

Changes file and line number reported by real compiler

Used for debugging purposes

Example:

error “Please fix <bits/endian.h>”

Natural Size on 64 Bit Machines

Rust :

i64 : signed integer $[-2^{63} \ldots 2^{63} - 1]$

u64 : unsigned integer $[0 \ldots , \ 2^{64} - 1]$

C, C++:

long long : signed integer $[-2^{63} \ldots 2^{63} - 1]$

unsigned long long : unsigned integer $[0 \ldots , \ 2^{64} - 1]$

Problematic! These languages existed before 64 bit machines

How can we use the preproceesor to fix C and C++?

New way for C

#include <stdint.h>

C:

int64_t: signed integer [-2^{63} … $2^{63} - 1$]

uint64_t: unsigned integer [0 … , $2^{64} - 1$]

New way for C++

#include <cstdint>

C++:

int64_t: signed integer [-2^{63} … $2^{63} - 1$]

uint64_t: unsigned integer [0 … , $2^{64} - 1$]

Defining variables

Assume the variable name is **my_integer** *and the machine size is 64 bits with no initial state*

Rust:

```
let my_integer : i64 ;
```

C:

```
#include <stdint.h>
    int64_t my_integer ;
```

C++:

```
#include <cstdint>
    int64_t my_integer ;
```

Variables with initial state

Assume the variable name is **my_integer** *and the machine size is 64 bits with initial state equal to 7*

Rust:

```
    let my_integer : i64 = 7 ;
```

C:

```
#include <stdint.h>
    int64_t my_integer = 7 ;
```

C++:

```
#include <cstdint>
    int64_t my_integer = 7 ;
```

Basic Variable Type

These type of variables are treated as **atomic** *or indivisible.*
Signed Integer types

Rust:

i8, i16, i32, i64

C:

#include <stdint.h>

int8_t, int16_t, int32_t, int64_t

signed char (equal to int8_t but deprecated)

C++:

#include <cstdint>

int8_t, int16_t, int32_t, int64_t, signed char

Basic Variable Type

These type of variables are treated as **atomic** *or indivisible.*
Unsigned Integer types

Rust:

u8, u16, u32, u64

C:

#include <stdint.h>

uint8_t, uint16_t, uint32_t, uint64_t

unsigned char (equal to uint8_t but deprecated)

C++:

#include <cstdint>

uint8_t, uint16_t, uint32_t, uint64_t, unsigned char

Floating Point Variables

Floating point numbers are numbers with a decimal point
Can be represented as mantissa x base ^{exponent}

Rust:

f32 *: single precision using 4 bytes*

f64 *: double precision using 8 bytes*

C, C++:

float : single precision using 4 bytes

double : double precision using 8 bytes

long double : may be 80 or 128 bits (compiler dependent)

Boolean Variables

Variable may either true or false

Rust:

bool : 1 byte storage

C:

#include <stdbool.h>

bool : 1 byte storage

C++: No include necessary

bool : 1 byte storage

Boolean Usage

All languages use **true** *and* **false** *keywords in lower case*

Rust:

```
let my_bool : bool = true ;
```

C:

```
#include <stdbool.h>
bool  my_bool = true ;
```

C++:

```
bool my_bool = false ;
```

Character Variables

Used as a single character in a "string" of characters

Rust:

char : storage space 4 bytes

C, C++:

char : storage space 1 byte

Let's review memory

Exercise: Using mempractice